

Project 09: Event Streaming & Analytics Platform

Difficulty: Advanced | Estimated Time: 3 Weeks

1. Project Overview and Goals

This project builds the database backend for an event streaming and analytics platform — similar to Mixpanel, Amplitude, or an internal analytics system. It covers high-volume event ingestion, user session tracking, funnel analysis, cohort retention, A/B testing, real-time dashboards, and alert systems. This is the most data-intensive project in the series and showcases PostgreSQL's capabilities as an analytical database.

Goals:

- Design a high-write event ingestion schema using table partitioning.
 - Implement funnel analysis queries for product analytics.
 - Build cohort retention analysis using window functions and CTEs.
 - Design an A/B test framework with statistical significance tracking.
 - Use JSONB for flexible event properties with GIN indexing.
 - Implement materialized views for dashboard caching.
 - Demonstrate PostgreSQL's analytical query capabilities at scale.
-

2. Learning Objectives

- Master table partitioning for time-series event data.
 - Write complex funnel queries using windowed LEAD/LAG.
 - Implement cohort retention matrices with pivot-style aggregations.
 - Use JSONB operators for property-based event filtering.
 - Build alert systems using threshold-based materialized views.
 - Practice `EXPLAIN ANALYZE` to optimize analytical queries.
 - Use parallel query hints for large dataset scans.
 - Understand the difference between append-only vs. mutable tables.
 - Write A/B test analysis including conversion rate confidence intervals.
-

3. Functional Requirements

- **Events:** Ingest named events with user ID, session, timestamp, and arbitrary JSONB properties.
 - **Users:** Track user profiles and first/last seen data.
 - **Sessions:** Group events into sessions with duration and page count.
 - **Funnels:** Define multi-step funnels and measure conversion at each step.
 - **Cohorts:** Group users by first-event date and track retention.
 - **A/B Tests:** Define experiments, assign users to variants, track outcomes.
 - **Dashboards:** Configurable metric tiles with SQL-backed queries.
 - **Alerts:** Threshold and anomaly alerts on key metrics.
 - **Segmentation:** Filter users by property conditions.
-

4. Non-Functional Requirements

- Events table partitioned by day (high write volume).
- Event IDs use UUID for distributed ingestion.

- Event table is append-only — no updates.
- JSONB properties indexed with GIN for fast filtering.
- Materialized views for all dashboard tiles, refreshed every 5 minutes.
- Retention queries must complete in under 3 seconds.

5. Complete Database Schema

```
-- =====
-- EVENT STREAMING & ANALYTICS PLATFORM - Complete Schema
-- PostgreSQL 15+
-- =====

CREATE SCHEMA IF NOT EXISTS analytics;
SET search_path = analytics, public;

-----

-- APPLICATIONS / PROJECTS
-----

CREATE TABLE apps (
    app_id          SERIAL PRIMARY KEY,
    api_key         UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE,
    name            VARCHAR(200) NOT NULL,
    description     TEXT,
    owner_email     VARCHAR(300),
    timezone        VARCHAR(50) NOT NULL DEFAULT 'UTC',
    is_active       BOOLEAN NOT NULL DEFAULT TRUE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-----

-- USERS
-----

CREATE TABLE users (
    user_id         BIGSERIAL PRIMARY KEY,
    app_id          INTEGER NOT NULL REFERENCES apps(app_id),
    external_id     VARCHAR(200) NOT NULL, -- your app's user ID
    anonymous_id    VARCHAR(100),         -- before login
    email           VARCHAR(300),
    name           VARCHAR(300),
    first_seen_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_seen_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    first_event_date DATE NOT NULL DEFAULT CURRENT_DATE,
    total_events    BIGINT NOT NULL DEFAULT 0,
    total_sessions  INTEGER NOT NULL DEFAULT 0,
    properties      JSONB NOT NULL DEFAULT '{}',
    UNIQUE (app_id, external_id)
);

CREATE INDEX idx_users_app      ON users (app_id, first_event_date);
CREATE INDEX idx_users_external ON users (app_id, external_id);
```

```

CREATE INDEX idx_users_props      ON users USING GIN (properties);

-----
-- EVENTS (append-only, partitioned by day)
-----

CREATE TABLE events (
    event_id      UUID NOT NULL DEFAULT gen_random_uuid(),
    app_id        INTEGER NOT NULL,
    user_id       BIGINT NOT NULL,
    session_id    VARCHAR(100),
    event_name     VARCHAR(200) NOT NULL,
    properties     JSONB NOT NULL DEFAULT '{}',
    device_type   VARCHAR(20) CHECK (device_type IN
('web', 'ios', 'android', 'server')),
    country       VARCHAR(100),
    city          VARCHAR(100),
    utm_source     VARCHAR(100),
    utm_medium     VARCHAR(100),
    utm_campaign   VARCHAR(200),
    received_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    sent_at       TIMESTAMPTZ
) PARTITION BY RANGE (received_at);

COMMENT ON TABLE events IS 'Append-only event ledger, partitioned by day';

-- Create partitions for recent dates
CREATE TABLE events_2024_10 PARTITION OF events
    FOR VALUES FROM ('2024-10-01') TO ('2024-11-01');
CREATE TABLE events_2024_11 PARTITION OF events
    FOR VALUES FROM ('2024-11-01') TO ('2024-12-01');
CREATE TABLE events_2024_12 PARTITION OF events
    FOR VALUES FROM ('2024-12-01') TO ('2025-01-01');
CREATE TABLE events_2025_01 PARTITION OF events
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
CREATE TABLE events_2025_q2 PARTITION OF events
    FOR VALUES FROM ('2025-04-01') TO ('2025-07-01');

-- Indexes on each partition (auto-inherited with PostgreSQL 11+)
CREATE INDEX idx_events_user      ON events (app_id, user_id, received_at DESC);
CREATE INDEX idx_events_name     ON events (app_id, event_name, received_at DESC);
CREATE INDEX idx_events_session  ON events (session_id, received_at);
CREATE INDEX idx_events_props    ON events USING GIN (properties);
CREATE INDEX idx_events_date     ON events (received_at);

-----
-- SESSIONS
-----

CREATE TABLE sessions (
    session_id     VARCHAR(100) NOT NULL,
    app_id         INTEGER NOT NULL,
    user_id        BIGINT NOT NULL REFERENCES users(user_id),
    started_at     TIMESTAMPTZ NOT NULL,

```

```

        ended_at          TIMESTAMPTZ,
        duration_seconds  INTEGER,
        event_count       INTEGER NOT NULL DEFAULT 0,
        device_type       VARCHAR(20),
        entry_page        VARCHAR(500),
        exit_page         VARCHAR(500),
        utm_source        VARCHAR(100),
        country           VARCHAR(100),
        PRIMARY KEY (session_id, app_id)
    ) PARTITION BY RANGE (started_at);

CREATE TABLE sessions_2024_q4 PARTITION OF sessions
    FOR VALUES FROM ('2024-10-01') TO ('2025-01-01');
CREATE TABLE sessions_2025 PARTITION OF sessions
    FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');

CREATE INDEX idx_sessions_user ON sessions (app_id, user_id, started_at DESC);

-----
-- FUNNELS
-----

CREATE TABLE funnels (
    funnel_id    SERIAL PRIMARY KEY,
    app_id       INTEGER NOT NULL REFERENCES apps(app_id),
    name        VARCHAR(200) NOT NULL,
    description  TEXT,
    window_hours INTEGER NOT NULL DEFAULT 168, -- 7 days default
    is_active   BOOLEAN NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE funnel_steps (
    step_id      SERIAL PRIMARY KEY,
    funnel_id    INTEGER NOT NULL REFERENCES funnels(funnel_id) ON DELETE CASCADE,
    step_number  SMALLINT NOT NULL,
    event_name   VARCHAR(200) NOT NULL,
    filters      JSONB NOT NULL DEFAULT '{}', -- optional property filters
    UNIQUE (funnel_id, step_number)
);

-----
-- A/B TESTS / EXPERIMENTS
-----

CREATE TABLE experiments (
    experiment_id SERIAL PRIMARY KEY,
    app_id       INTEGER NOT NULL REFERENCES apps(app_id),
    name        VARCHAR(200) NOT NULL,
    description  TEXT,
    status      VARCHAR(20) NOT NULL DEFAULT 'Draft'
    CHECK (status IN
('Draft', 'Running', 'Paused', 'Concluded', 'Archived')),
    hypothesis   TEXT,

```

```

    primary_metric VARCHAR(200),
    started_at    TIMESTAMPTZ,
    ended_at      TIMESTAMPTZ,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE experiment_variants (
    variant_id     SERIAL PRIMARY KEY,
    experiment_id  INTEGER NOT NULL REFERENCES experiments(experiment_id),
    name           VARCHAR(100) NOT NULL,
    description    TEXT,
    traffic_pct    NUMERIC(5,2) NOT NULL DEFAULT 50.00 CHECK (traffic_pct BETWEEN 0
AND 100),
    is_control     BOOLEAN NOT NULL DEFAULT FALSE,
    UNIQUE (experiment_id, name)
);

CREATE TABLE experiment_assignments (
    assignment_id  BIGSERIAL PRIMARY KEY,
    experiment_id  INTEGER NOT NULL REFERENCES experiments(experiment_id),
    variant_id     INTEGER NOT NULL REFERENCES experiment_variants(variant_id),
    user_id        BIGINT NOT NULL REFERENCES users(user_id),
    assigned_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (experiment_id, user_id)
);

CREATE INDEX idx_exp_assignments ON experiment_assignments (experiment_id,
variant_id);

-- -----
-- ALERTS
-- -----

CREATE TABLE alert_rules (
    rule_id        SERIAL PRIMARY KEY,
    app_id         INTEGER NOT NULL REFERENCES apps(app_id),
    name           VARCHAR(200) NOT NULL,
    metric_query   TEXT NOT NULL,    -- SQL template
    condition      VARCHAR(10) NOT NULL CHECK (condition IN ('>', '<', '>=', '<=',
='')),
    threshold      NUMERIC,
    window_min     INTEGER NOT NULL DEFAULT 60,
    severity       VARCHAR(10) NOT NULL DEFAULT 'Medium'
CHECK (severity IN ('Low', 'Medium', 'High', 'Critical')),
    is_active      BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE alert_firings (
    firing_id      BIGSERIAL PRIMARY KEY,
    rule_id        INTEGER NOT NULL REFERENCES alert_rules(rule_id),
    metric_value    NUMERIC NOT NULL,
    fired_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    resolved_at    TIMESTAMPTZ,

```

```

        notes          TEXT
    );

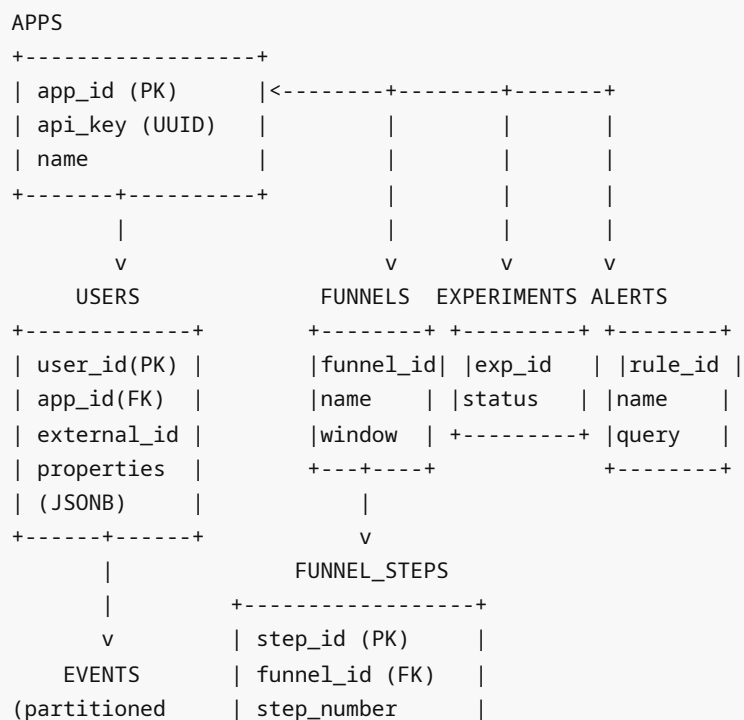
-- -----
-- DASHBOARD TILES
-- -----

CREATE TABLE dashboards (
    dashboard_id SERIAL PRIMARY KEY,
    app_id        INTEGER NOT NULL REFERENCES apps(app_id),
    name          VARCHAR(200) NOT NULL,
    is_default    BOOLEAN NOT NULL DEFAULT FALSE,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE dashboard_tiles (
    tile_id        SERIAL PRIMARY KEY,
    dashboard_id   INTEGER NOT NULL REFERENCES dashboards(dashboard_id),
    title          VARCHAR(200) NOT NULL,
    chart_type     VARCHAR(20) CHECK (chart_type IN
('line','bar','pie','number','table','heatmap')),
    metric_query   TEXT NOT NULL,
    position_x     SMALLINT,
    position_y     SMALLINT,
    width          SMALLINT DEFAULT 4,
    height         SMALLINT DEFAULT 3
);

```

6. ASCII ER Diagram



```

by received_at) | event_name |
+-----+-----+
| event_id(UUID) |
| app_id         | EXPERIMENT_VARIANTS
| user_id(FK)    | +-----+
| event_name     | | variant_id (PK) |
| properties     | | experiment_id(FK) |
| (JSONB)        | | traffic_pct      |
| received_at    | | is_control       |
+-----+-----+ +-----+-----+
|               |               |
v               v
SESSIONS        EXPERIMENT_ASSIGNMENTS
(partitioned)   +-----+
+-----+       | assignment_id(PK) |
| session_id | | experiment_id(FK) |
| user_id(FK) | | variant_id (FK)   |
| duration   | | user_id (FK)     |
+-----+       +-----+
DASHBOARDS      ALERT_FIRINGS
+-----+       +-----+
|dashboard_id| | firing_id(PK) |
|name        | | rule_id (FK)  |
+-----+       | metric_value |
|            | | fired_at      |
v            +-----+
DASHBOARD_TILES
+-----+
| tile_id (PK) |
| dashboard_id(FK) |
| metric_query |
+-----+

```

7. Sample Data Scripts

```
SET search_path = analytics, public;

-- App
INSERT INTO apps (name, description, owner_email) VALUES
('SaaS Product', 'Main product analytics', 'analytics@company.com');

-- Users
INSERT INTO users (app_id, external_id, email, name, first_event_date, properties)
VALUES
(1, 'user_001', 'alice@co.com', 'Alice Chen', '2024-10-01',
 '{"plan":"pro","company":"Acme","signup_source":"google"}'::JSONB),
(1, 'user_002', 'bob@co.com', 'Bob Martinez', '2024-10-05',
 '{"plan":"free","company":"BlueSky","signup_source":"direct"}'::JSONB),
(1, 'user_003', 'carol@co.com', 'Carol Lee', '2024-10-10',
```

```

    '{"plan":"pro","company":"GreenLeaf","signup_source":"referral"}'::JSONB),
(1, 'user_004', 'david@co.com', 'David Kim', '2024-11-01',
 '{"plan":"free","company":"NovaCo","signup_source":"linkedin"}'::JSONB),
(1, 'user_005', 'emma@co.com', 'Emma Garcia', '2024-11-15',
 '{"plan":"pro","company":"TechStart","signup_source":"google"}'::JSONB);

-- Events (onboarding funnel events)
INSERT INTO events (app_id, user_id, session_id, event_name, properties,
device_type, received_at) VALUES
(1,1,'sess_001','page_view', '{"page":"/home"}'::JSONB, 'web', NOW()-INTERVAL
'30 days'),
(1,1,'sess_001','signup_start', '{"source":"hero_button"}'::JSONB, 'web', NOW()-
INTERVAL '30 days' + INTERVAL '2 min'),
(1,1,'sess_001','signup_complete', '{"plan":"pro"}'::JSONB, 'web', NOW()-INTERVAL
'30 days' + INTERVAL '5 min'),
(1,1,'sess_001','onboarding_step_1', '{"step":"profile"}'::JSONB, 'web', NOW()-
INTERVAL '30 days' + INTERVAL '8 min'),
(1,1,'sess_001','onboarding_complete', '{"time_to_complete":420}'::JSONB, 'web',
NOW()-INTERVAL '30 days' + INTERVAL '15 min'),

(1,2,'sess_002','page_view', '{"page":"/home"}'::JSONB, 'web', NOW()-INTERVAL
'25 days'),
(1,2,'sess_002','signup_start', '{"source":"nav_button"}'::JSONB, 'web', NOW()-
INTERVAL '25 days' + INTERVAL '1 min'),
(1,2,'sess_002','signup_complete', '{"plan":"free"}'::JSONB, 'web', NOW()-INTERVAL
'25 days' + INTERVAL '3 min'),
-- Bob abandons onboarding
(1,3,'sess_003','page_view', '{"page":"/home"}'::JSONB, 'ios', NOW()-INTERVAL
'20 days'),
(1,3,'sess_003','signup_start', '{"source":"mobile_cta"}'::JSONB, 'ios', NOW()-
INTERVAL '20 days' + INTERVAL '1 min'),
(1,3,'sess_003','signup_complete', '{"plan":"pro"}'::JSONB, 'ios', NOW()-INTERVAL
'20 days' + INTERVAL '4 min'),
(1,3,'sess_003','onboarding_step_1', '{"step":"profile"}'::JSONB, 'ios', NOW()-
INTERVAL '20 days' + INTERVAL '6 min'),

(1,3,'sess_003','onboarding_complete', '{"time_to_complete":380}'::JSONB, 'ios', NOW()-
INTERVAL '20 days' + INTERVAL '12 min'),

-- Feature usage events

(1,1,'sess_010','feature_used', '{"feature":"report_builder","plan":"pro"}'::JSONB, 'web',
INTERVAL '5 days'),

(1,1,'sess_010','feature_used', '{"feature":"export_csv","plan":"pro"}'::JSONB, 'web', NOW
INTERVAL '5 days' + INTERVAL '30 min'),

(1,3,'sess_011','feature_used', '{"feature":"dashboard","plan":"pro"}'::JSONB, 'ios', NOW(
INTERVAL '3 days'),

(1,5,'sess_012','upgrade_click', '{"from_plan":"free","to_plan":"pro"}'::JSONB, 'web', NOW
INTERVAL '2 days');

```



```

-- Funnels
INSERT INTO funnels (app_id, name, description, window_hours) VALUES
  (1, 'Signup to Activated', 'User signs up and completes onboarding', 72),
  (1, 'Free to Pro Upgrade', 'User upgrades from free to pro plan', 720);

INSERT INTO funnel_steps (funnel_id, step_number, event_name) VALUES
  (1, 1, 'page_view'),
  (1, 2, 'signup_start'),
  (1, 3, 'signup_complete'),
  (1, 4, 'onboarding_step_1'),
  (1, 5, 'onboarding_complete');

-- Experiments
INSERT INTO experiments (app_id, name, hypothesis, primary_metric, status,
started_at) VALUES
  (1, 'Onboarding Flow v2', 'Simplified onboarding will increase activation rate by
10%',
  'onboarding_complete', 'Running', NOW()-INTERVAL '14 days');

INSERT INTO experiment_variants (experiment_id, name, traffic_pct, is_control)
VALUES
  (1, 'Control', 50, TRUE),
  (1, 'Simplified', 50, FALSE);

INSERT INTO experiment_assignments (experiment_id, variant_id, user_id) VALUES
  (1, 1, 1), (1, 1, 2), (1, 2, 3), (1, 2, 4), (1, 2, 5);

```

8. Core SQL Queries

```

SET search_path = analytics, public;

-- -----
-- Q1: Daily Active Users (DAU) last 30 days
-- -----
SELECT
  DATE(e.received_at)           AS date,
  COUNT(DISTINCT e.user_id)    AS dau
FROM events e
WHERE e.app_id = 1
  AND e.received_at >= CURRENT_DATE - 30
GROUP BY DATE(e.received_at)
ORDER BY date;

-- -----
-- Q2: Weekly Active Users (WAU) and Monthly (MAU)
-- -----
SELECT
  'DAU' AS metric,
  ROUND(AVG(daily_count), 0) AS avg_value

```

```

FROM (
    SELECT DATE(received_at), COUNT(DISTINCT user_id) AS daily_count
    FROM events WHERE app_id = 1 AND received_at >= NOW() - INTERVAL '30 days'
    GROUP BY DATE(received_at)
) d

UNION ALL

SELECT 'WAU', COUNT(DISTINCT user_id)
FROM events WHERE app_id = 1 AND received_at >= NOW() - INTERVAL '7 days'

UNION ALL

SELECT 'MAU', COUNT(DISTINCT user_id)
FROM events WHERE app_id = 1 AND received_at >= NOW() - INTERVAL '30 days';

-- -----
-- Q3: Funnel analysis - onboarding conversion
-- -----

WITH funnel_events AS (
    SELECT
        e.user_id,
        e.event_name,
        MIN(e.received_at) AS first_occurrence
    FROM events e
    JOIN funnel_steps fs ON e.event_name = fs.event_name
    WHERE e.app_id = 1
        AND fs.funnel_id = 1
        AND e.received_at >= NOW() - INTERVAL '30 days'
    GROUP BY e.user_id, e.event_name
),
step_counts AS (
    SELECT
        fs.step_number,
        fs.event_name,
        COUNT(DISTINCT fe.user_id) AS users_at_step
    FROM funnel_steps fs
    LEFT JOIN funnel_events fe ON fs.event_name = fe.event_name
    WHERE fs.funnel_id = 1
    GROUP BY fs.step_number, fs.event_name
)
SELECT
    step_number,
    event_name,
    users_at_step,
    FIRST_VALUE(users_at_step) OVER (ORDER BY step_number) AS entered_funnel,
    ROUND(100.0 * users_at_step
        / FIRST_VALUE(users_at_step) OVER (ORDER BY step_number), 1) AS
overall_conversion_pct,
    ROUND(100.0 * users_at_step
        / LAG(users_at_step) OVER (ORDER BY step_number), 1) AS
step_conversion_pct

```

```

FROM step_counts
ORDER BY step_number;

-- -----
-- Q4: Cohort retention matrix
-- -----

WITH cohorts AS (
    SELECT
        user_id,
        DATE_TRUNC('week', first_event_date)::DATE AS cohort_week
    FROM users WHERE app_id = 1
),
activity AS (
    SELECT
        e.user_id,
        DATE_TRUNC('week', e.received_at)::DATE AS active_week
    FROM events e WHERE e.app_id = 1
    GROUP BY e.user_id, DATE_TRUNC('week', e.received_at)
),
cohort_activity AS (
    SELECT
        c.cohort_week,
        (a.active_week - c.cohort_week) / 7 AS weeks_after,
        COUNT(DISTINCT c.user_id) AS cohort_size,
        COUNT(DISTINCT a.user_id) AS retained_users
    FROM cohorts c
    LEFT JOIN activity a ON c.user_id = a.user_id
    AND a.active_week >= c.cohort_week
    GROUP BY c.cohort_week, weeks_after
)
SELECT
    cohort_week,
    MAX(cohort_size) FILTER (WHERE weeks_after = 0) AS cohort_size,
    ROUND(100.0 * MAX(retained_users) FILTER (WHERE weeks_after = 0) /
NULLIF(MAX(cohort_size) FILTER (WHERE weeks_after = 0), 0), 1) AS week_0_pct,
    ROUND(100.0 * MAX(retained_users) FILTER (WHERE weeks_after = 1) /
NULLIF(MAX(cohort_size) FILTER (WHERE weeks_after = 0), 0), 1) AS week_1_pct,
    ROUND(100.0 * MAX(retained_users) FILTER (WHERE weeks_after = 2) /
NULLIF(MAX(cohort_size) FILTER (WHERE weeks_after = 0), 0), 1) AS week_2_pct,
    ROUND(100.0 * MAX(retained_users) FILTER (WHERE weeks_after = 4) /
NULLIF(MAX(cohort_size) FILTER (WHERE weeks_after = 0), 0), 1) AS week_4_pct
FROM cohort_activity
GROUP BY cohort_week
ORDER BY cohort_week;

-- -----
-- Q5: A/B test results with conversion rates
-- -----

WITH variant_totals AS (
    SELECT
        ev.variant_id,
        ev2.name AS variant_name,

```

```

        ev2.is_control,
        COUNT(DISTINCT ea.user_id) AS users_assigned
    FROM experiment_assignments ea
    JOIN experiment_variants ev ON ea.variant_id = ev.variant_id
    JOIN experiment_variants ev2 ON ea.variant_id = ev2.variant_id
    WHERE ea.experiment_id = 1
    GROUP BY ev.variant_id, ev2.name, ev2.is_control
),
conversions AS (
    SELECT
        ea.variant_id,
        COUNT(DISTINCT ea.user_id) AS converted_users
    FROM experiment_assignments ea
    JOIN events e ON ea.user_id = e.user_id
    AND e.event_name = 'onboarding_complete'
    AND e.received_at >= (SELECT started_at FROM experiments WHERE experiment_id
= 1)
    WHERE ea.experiment_id = 1
    GROUP BY ea.variant_id
)
SELECT
    vt.variant_name,
    vt.is_control,
    vt.users_assigned,
    COALESCE(c.converted_users, 0) AS conversions,
    ROUND(100.0 * COALESCE(c.converted_users, 0) / NULLIF(vt.users_assigned, 0), 2)
AS conversion_rate_pct,
    ROUND(100.0 * COALESCE(c.converted_users, 0) / NULLIF(vt.users_assigned, 0)
    - MAX(100.0 * COALESCE(c.converted_users, 0) / NULLIF(vt.users_assigned, 0))
    FILTER (WHERE vt.is_control = TRUE) OVER (), 2) AS delta_vs_control_pct
FROM variant_totals vt
LEFT JOIN conversions c ON vt.variant_id = c.variant_id
ORDER BY vt.is_control DESC;

-- -----
-- Q6: Top events by frequency (last 7 days)
-- -----

SELECT
    event_name,
    COUNT(*) AS occurrences,
    COUNT(DISTINCT user_id) AS unique_users,
    ROUND(COUNT(*) / 7.0, 1) AS avg_per_day,
    MAX(received_at) AS last_seen
FROM events
WHERE app_id = 1
    AND received_at >= NOW() - INTERVAL '7 days'
GROUP BY event_name
ORDER BY occurrences DESC;

-- -----
-- Q7: User segmentation by JSONB properties
-- -----

```

```

SELECT
    properties->>'plan'          AS plan,
    properties->>'signup_source' AS signup_source,
    COUNT(*)                    AS user_count,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct
FROM users
WHERE app_id = 1
GROUP BY properties->>'plan', properties->>'signup_source'
ORDER BY user_count DESC;

```

```

-----
-- Q8: Event property distribution
-----

```

```

SELECT
    properties->>'feature'      AS feature,
    COUNT(*)                   AS uses,
    COUNT(DISTINCT user_id)    AS unique_users,
    COUNT(DISTINCT session_id) AS sessions,
    ROUND(COUNT(*) * 1.0 / COUNT(DISTINCT user_id), 1) AS uses_per_user
FROM events
WHERE app_id = 1
    AND event_name = 'feature_used'
    AND received_at >= NOW() - INTERVAL '30 days'
GROUP BY properties->>'feature'
ORDER BY uses DESC;

```

```

-----
-- Q9: Session depth analysis
-----

```

```

SELECT
    event_count          AS events_per_session,
    COUNT(*)             AS session_count,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct
FROM sessions
WHERE app_id = 1
    AND started_at >= NOW() - INTERVAL '30 days'
GROUP BY event_count
ORDER BY event_count;

```

```

-----
-- Q10: Time-to-convert funnel timing analysis
-----

```

```

WITH user_steps AS (
    SELECT
        user_id,
        MIN(received_at) FILTER (WHERE event_name = 'signup_complete') AS
signup_time,
        MIN(received_at) FILTER (WHERE event_name = 'onboarding_complete') AS
activated_time
    FROM events
    WHERE app_id = 1
    GROUP BY user_id
)

```

```

)
SELECT
    CASE
        WHEN EXTRACT(EPOCH FROM (activated_time - signup_time)) / 3600 < 1 THEN '<
1 hour'
        WHEN EXTRACT(EPOCH FROM (activated_time - signup_time)) / 3600 < 24 THEN '1-
24 hours'
        WHEN EXTRACT(EPOCH FROM (activated_time - signup_time)) / 3600 < 72 THEN '1-
3 days'
        WHEN EXTRACT(EPOCH FROM (activated_time - signup_time)) / 3600 < 168 THEN
'3-7 days'
        ELSE '> 7 days'
    END AS time_to_activate,
    COUNT(*) AS users,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct
FROM user_steps
WHERE signup_time IS NOT NULL AND activated_time IS NOT NULL
GROUP BY 1
ORDER BY MIN(EXTRACT(EPOCH FROM (activated_time - signup_time)));

-----
-- Q11: Rolling 7-day average events
-----
SELECT
    date,
    dau,
    ROUND(AVG(dau) OVER (ORDER BY date ROWS 6 PRECEDING), 0) AS rolling_7d_avg
FROM (
    SELECT DATE(received_at) AS date, COUNT(DISTINCT user_id) AS dau
    FROM events WHERE app_id = 1
    GROUP BY DATE(received_at)
) daily
ORDER BY date;

-----
-- Q12: Power users (top 10% by event count)
-----
WITH user_activity AS (
    SELECT
        u.external_id,
        u.name,
        u.properties->>'plan' AS plan,
        COUNT(e.event_id) AS event_count,
        COUNT(DISTINCT DATE(e.received_at)) AS active_days,
        NTILE(10) OVER (ORDER BY COUNT(e.event_id)) AS decile
    FROM users u
    LEFT JOIN events e ON u.user_id = e.user_id
    AND e.received_at >= NOW() - INTERVAL '30 days'
    WHERE u.app_id = 1
    GROUP BY u.user_id, u.external_id, u.name, u.properties
)
SELECT *

```

```

FROM user_activity
WHERE decile = 10 -- top 10%
ORDER BY event_count DESC;

-----
-- Q13: Event occurrence within N minutes of another (correlation)
-----

SELECT
    e1.event_name AS event_a,
    e2.event_name AS event_b,
    COUNT(*)      AS co_occurrence_count,
    ROUND(AVG(EXTRACT(EPOCH FROM (e2.received_at - e1.received_at)) / 60), 1) AS
avg_minutes_between
FROM events e1
JOIN events e2 ON e1.user_id = e2.user_id
    AND e1.session_id = e2.session_id
    AND e2.received_at > e1.received_at
    AND e2.received_at <= e1.received_at + INTERVAL '30 minutes'
    AND e1.event_name <> e2.event_name
WHERE e1.app_id = 1
    AND e1.received_at >= NOW() - INTERVAL '30 days'
GROUP BY e1.event_name, e2.event_name
HAVING COUNT(*) > 2
ORDER BY co_occurrence_count DESC
LIMIT 20;

```

9. Stored Procedures / Functions

```

-----
-- FUNCTION 1: Ingest a batch of events (high-performance)
-----

CREATE OR REPLACE FUNCTION analytics.ingest_events(
    p_events JSONB -- array of event objects
)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_count INTEGER := 0;
    v_event JSONB;
    v_uid BIGINT;
BEGIN
    FOR v_event IN SELECT jsonb_array_elements(p_events)
    LOOP
        -- Upsert user
        INSERT INTO analytics.users (app_id, external_id, email, name, properties)
        VALUES (
            (v_event->>'app_id')::INTEGER,
            v_event->>'user_id',
            v_event->>'email',
            v_event->>'name',

```

```

        COALESCE(v_event->'user_properties', '{}')
    )
ON CONFLICT (app_id, external_id) DO UPDATE
SET last_seen_at = NOW(),
    total_events = users.total_events + 1
RETURNING user_id INTO v_uid;

-- Insert event
INSERT INTO analytics.events (
    app_id, user_id, session_id, event_name, properties,
    device_type, country, utm_source, utm_campaign, received_at
) VALUES (
    (v_event->>'app_id')::INTEGER,
    v_uid,
    v_event->>'session_id',
    v_event->>'event_name',
    COALESCE(v_event->'properties', '{}'),
    v_event->>'device_type',
    v_event->>'country',
    v_event->>'utm_source',
    v_event->>'utm_campaign',
    COALESCE((v_event->>'received_at')::TIMESTAMPTZ, NOW())
);

v_count := v_count + 1;
END LOOP;

RETURN v_count;
END;
$$;

-----
-- FUNCTION 2: Compute funnel metrics for a given period
-----

CREATE OR REPLACE FUNCTION analytics.get_funnel_metrics(
    p_funnel_id INTEGER,
    p_start     TIMESTAMPTZ,
    p_end       TIMESTAMPTZ
)
RETURNS TABLE (
    step_number  SMALLINT,
    event_name   VARCHAR,
    users_entered BIGINT,
    conversion_pct NUMERIC,
    drop_off_pct  NUMERIC
)
LANGUAGE plpgsql AS $$
DECLARE
    v_funnel analytics.funnels%ROWTYPE;
BEGIN
    SELECT * INTO v_funnel FROM analytics.funnels WHERE funnel_id = p_funnel_id;
    IF NOT FOUND THEN RAISE EXCEPTION 'Funnel % not found', p_funnel_id; END IF;

```



```

RETURN QUERY
WITH step_users AS (
    SELECT
        fs.step_number,
        fs.event_name,
        COUNT(DISTINCT e.user_id) AS users
    FROM analytics.funnel_steps fs
    LEFT JOIN analytics.events e ON fs.event_name = e.event_name
        AND e.received_at BETWEEN p_start AND p_end
    WHERE fs.funnel_id = p_funnel_id
    GROUP BY fs.step_number, fs.event_name
)
SELECT
    su.step_number,
    su.event_name,
    su.users,
    ROUND(100.0 * su.users / NULLIF(FIRST_VALUE(su.users) OVER (ORDER BY
su.step_number), 0), 1),
    ROUND(100.0 * (LAG(su.users) OVER (ORDER BY su.step_number) - su.users)
        / NULLIF(LAG(su.users) OVER (ORDER BY su.step_number), 0), 1)
    FROM step_users su
    ORDER BY su.step_number;
END;
$$;

```

10. Triggers

```

-- TRIGGER 1: Update user last_seen_at on every event
CREATE OR REPLACE FUNCTION analytics.trg_update_user_last_seen()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    UPDATE analytics.users
    SET last_seen_at = NEW.received_at,
        total_events = total_events + 1
    WHERE user_id = NEW.user_id
        AND (last_seen_at IS NULL OR last_seen_at < NEW.received_at);
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_events_update_user
AFTER INSERT ON analytics.events
FOR EACH ROW EXECUTE FUNCTION analytics.trg_update_user_last_seen();

-- TRIGGER 2: Prevent updates on events table
CREATE OR REPLACE FUNCTION analytics.trg_events_readonly()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'Events table is append-only. No updates or deletes permitted.';

```

```

END;
$$;

CREATE TRIGGER trg_events_no_update
    BEFORE UPDATE OR DELETE ON analytics.events
    FOR EACH ROW EXECUTE FUNCTION analytics.trg_events_readonly();

```

11. Performance Optimization

```

-- Materialized view for daily KPIs
CREATE MATERIALIZED VIEW analytics.mv_daily_kpis AS
SELECT
    DATE(received_at)           AS date,
    app_id,
    COUNT(*)                   AS total_events,
    COUNT(DISTINCT user_id)    AS dau,
    COUNT(DISTINCT session_id) AS sessions,
    COUNT(*) FILTER (WHERE event_name = 'signup_complete') AS signups,
    COUNT(*) FILTER (WHERE event_name = 'onboarding_complete') AS activations
FROM analytics.events
WHERE received_at >= CURRENT_DATE - 90
GROUP BY DATE(received_at), app_id
WITH DATA;

CREATE UNIQUE INDEX ON analytics.mv_daily_kpis (date, app_id);

-- BRIN index on time-series events
CREATE INDEX idx_events_brin_time ON analytics.events USING BRIN (received_at);

```

12. Extensions Used

```

CREATE EXTENSION IF NOT EXISTS pgcrypto;           -- UUID generation
CREATE EXTENSION IF NOT EXISTS pg_stat_statements; -- Query performance tracking
CREATE EXTENSION IF NOT EXISTS pg_cron;             -- Scheduled materialied view refresh

-- Refresh every 5 minutes
SELECT cron.schedule('refresh-kpis', '* /5 * * * *',
    'REFRESH MATERIALIZED VIEW CONCURRENTLY analytics.mv_daily_kpis');

-- Auto-create daily partitions (requires pg_cron)
SELECT cron.schedule('create-event-partition', '50 23 * * *', $$
    SELECT analytics.create_event_partition(CURRENT_DATE + 1);
    $$);

```

13. Testing Guide

```

-- TEST 1: Batch event ingestion
SELECT analytics.ingest_events('[

{"app_id":1,"user_id":"user_006","event_name":"page_view","session_id":"sess_100",
  "properties":{"page":"/pricing"},"device_type":"web","received_at":"2025-01-
15T10:00:00Z"},

{"app_id":1,"user_id":"user_006","event_name":"upgrade_click","session_id":"sess_100",
  "properties":{"plan":"pro"},"device_type":"web","received_at":"2025-01-
15T10:05:00Z"}
]'::JSONB);

-- TEST 2: Funnel analysis
SELECT * FROM analytics.get_funnel_metrics(1, NOW() - INTERVAL '30 days', NOW());

-- TEST 3: Cohort retention (Q4) - verify matrix shape
-- Should show week_0 = 100%, week_1 declining

-- TEST 4: A/B test results (Q5)
-- Verify conversion rates differ between variants

-- TEST 5: JSONB property filtering
SELECT * FROM analytics.users WHERE properties @> '{"plan":"pro"}'::JSONB;

-- TEST 6: Prevent event update
UPDATE analytics.events SET event_name = 'hacked' WHERE user_id = 1;
-- Should raise error

-- TEST 7: Partition pruning verification
EXPLAIN SELECT * FROM analytics.events
WHERE received_at >= '2024-12-01' AND received_at < '2025-01-01';
-- Should show only events_2024_12 partition in plan

```

14. Extension Challenges

- Real-Time Streaming Ingestion:** Integrate with a message broker by implementing a `pg_notify` - based event router. Write a Python or Node.js consumer that listens for notification events and processes the analytics pipeline in near-real-time.
- Machine Learning Feature Store:** Add a `feature_vectors` table that computes ML features per user (event count by type, recency, frequency, monetary value for RFM scoring). Schedule weekly feature refresh with `pg_cron`.
- Custom Dashboard Builder:** Extend `dashboard_tiles` with a safe parameterized query executor. Write a PL/pgSQL function that executes stored metric queries with date range substitution and returns results as JSONB arrays for API consumption.
- Anomaly Detection:** Implement Z-score anomaly detection. For each metric, compute rolling mean and standard deviation. Flag days where the value is more than 2 standard deviations from the mean. Auto-fire alert events.

5. **Event Replay and Backfill:** Design a mechanism to reprocess historical events through new funnel definitions without duplicating data. Use a `funnel_cache` table to store pre-computed results per user per funnel per week.

15. What You Learned

Skill	Demonstrated By
Append-only event table	Trigger blocks UPDATE/DELETE on events
Range partitioning	Events partitioned by month for scan pruning
JSONB GIN indexing	<code>properties @></code> operator for property filters
Funnel analysis	Step-by-step conversion with window functions
Cohort retention matrix	Conditional aggregation pivot by week offset
NTILE for power users	Top decile identification
A/B test analysis	Conversion rate delta vs. control group
BRIN indexes	Time-ordered events benefit from BRIN
Materialized views	Daily KPI dashboard cached and scheduled
Batch ingestion	<code>ingest_events()</code> handles array of event objects